ESCUELA DE INGENIERÍA INFORMÁTICA



PONTIFICIA UNIVERSIDAD CATÓLICA DE VALPARAÍSO

REPORTE PROYECTO VIDEOJUEGO

GIULLIANO MARTÍNEZ
PATRICIO HERNÁNDEZ
ETHAN TRONCOSO

INF2236 PROGRAMACIÓN AVANZADA
13/11/2026

GM-1 Explicación del juego desde la perspectiva del jugador

Aplicación al Proyecto

Descripción general

Órbita Cero es un juego arcade espacial 2D. Eres quien permanece de guardia en el corredor de evacuación Órbita Cero: mientras resistas, la ruta continúa abierta. Para sostener esa defensa debes destruir las naves hostiles de cada ronda y evitar sus choques y disparos.

La partida comienza con 3 vidas, 0 puntos y 10 naves enemigas. Al eliminarlas se avanza de ronda: la cantidad de enemigos aumenta de dos en dos y su velocidad base aumenta en una unidad por eje en cada nueva ronda.

Tu objetivo es mantener abierto el corredor durante tantas rondas como puedas. Cada oleada eliminada permite que la defensa continúe; destruir enemigos aumenta tu puntuación y te da la oportunidad de superar el récord de la sesión.

Objetivo del jugador

Como último piloto de guardia, debes despejar cada oleada sin agotar las vidas de tu nave. La misión no tiene un final prefijado: el desafío es sostener la ruta abierta el mayor tiempo posible.

Cada NaveCaza destruida entrega 15 puntos y cada NaveArtillera, 35. Cuando ya no quedan enemigos y el jugador sigue vivo, comienza automáticamente la siguiente ronda.

Las vidas restantes y la puntuación obtenida se mantienen entre rondas, si la nave pierde todas las vidas, la partida termina y se muestra Game Over.

Narrativa

La evacuación atraviesa el corredor Órbita Cero. Las naves enemigas lo invaden en oleadas y tú eres la última guardia que aún puede mantener libre el paso. Al comenzar solo cuentas con tres vidas: cada choque o proyectil que atraviesa tu defensa acerca el cierre de la ruta. Cuando eliminas a todos los enemigos, el corredor gana un respiro y llega una nueva oleada, más numerosa y veloz. Si pierdes la última vida, termina tu guardia; desde la pantalla final puedes iniciar otra defensa.

Elementos principales del juego

Nave del jugador

Es el elemento controlado por el jugador.
La nave puede hacer:

- Desplazarse en todas las direcciones.

- Apuntar usando la dirección del ratón.
- Rotar hacia el punto que está apuntando.
- Disparar proyectiles
- Recibir daño por colisiones con naves enemigas o por sus proyectiles.

Naves enemigas

Constituyen los adversarios del juego: NaveCaza resiste un impacto y dispara una bala; NaveArtillera resiste dos y dispara tres.
Se desplazan continuamente dentro del escenario y rebotan:

- Contra los límites del escenario de la ronda
- Con ellas mismas

Al ser destruidas otorgan 15 puntos (NaveCaza) o 35 (NaveArtillera). Cada impacto de un proyectil del jugador reproduce un sonido de explosión.

Proyectiles

El jugador dispara hacia el puntero del ratón: una bala por activación y tres desde la tercera ronda. Al impactar, el proyectil desaparece y el enemigo pierde un punto de vida; solo se elimina cuando su vida llega a cero. Los enemigos también disparan hacia el jugador. Los proyectiles se retiran al alcanzar los límites del área.

Sistema de vidas y daño

La partida comienza con 3 vidas, cuando la nave colisiona con una nave enemiga o recibe uno de sus proyectiles, pierde una vida si no está protegida.

Después de recibir daño, la nave obtiene aproximadamente 1 segundo de protección, durante el cual no puede perder vidas. Visualmente durante ese periodo la nave aparece semitransparente.

Esto evita que en solamente un choque pierda inmediatamente varias vidas.

Puntaje

El jugador obtiene puntos por destruir naves enemigas. 15 puntos por NaveCaza y 35 por NaveArtillera destruida. Durante la partida se muestran dos valores:

- Puntaje
- Récord

Puntaje: corresponde a la puntuación de la partida actual.
Récord: representa la mayor puntuación registrada al terminar una partida durante la ejecución actual del juego.

Rondas

Cada ronda contiene una determinada cantidad de naves enemigas, la primera ronda se inicia con:

- Ronda: 1
- Naves enemigas: 10
- Vidas: 3
- Puntaje: 0

Cuando todas las naves enemigas son destruidas y quedan vidas, se pasa a la siguiente ronda conservando vidas y puntaje.

Controles

Acción:	Control:
Mover hacia arriba	w
Mover hacia abajo	s
Mover hacia izquierda	a
Mover hacia derecha	d
Apuntar	movimiento del ratón
Disparar	Clic izquierdo

HUD durante la partida

En la parte inferior de la pantalla el jugador puede observar:
Vidas: 3 Ronda: 1 Récord: 0 Puntaje: 0

Esto entrega información constante sobre:

- Vidas restantes
- Ronda actual
- Mejor puntuación registrada
- Puntuación de la partida.

Evidencia

(Menu principal)

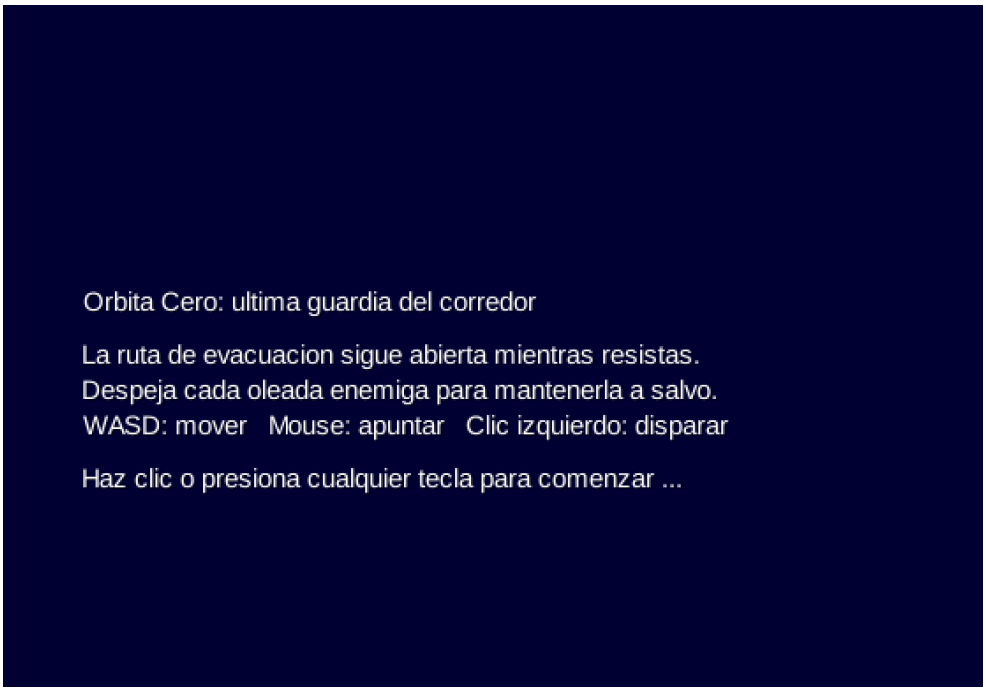


Figura 1. Inicio del juego; la misión aparece.

(Ronda inicial)

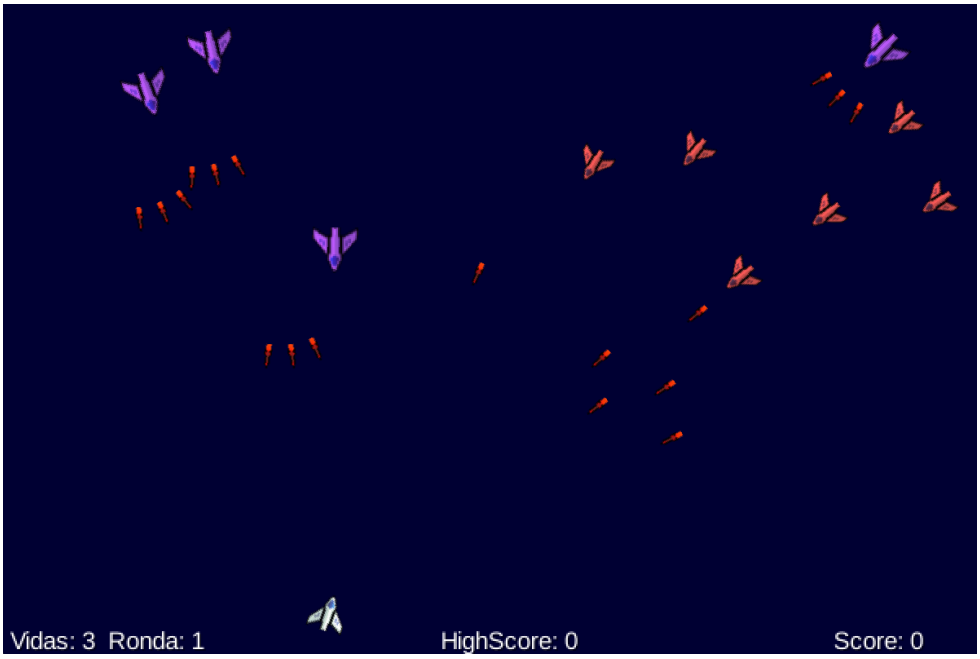


Figura 2. Primera oleada: el piloto comienza la defensa con tres vidas y diez enemigos.

(Ronda 3 o superior)

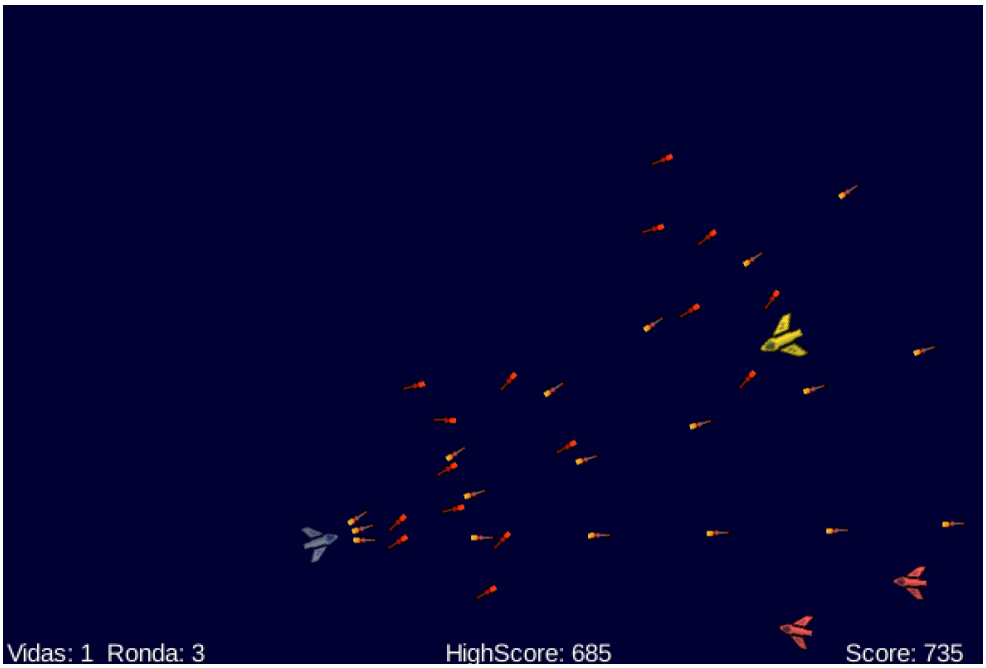


Figura 3. Ronda 3

(Game over)

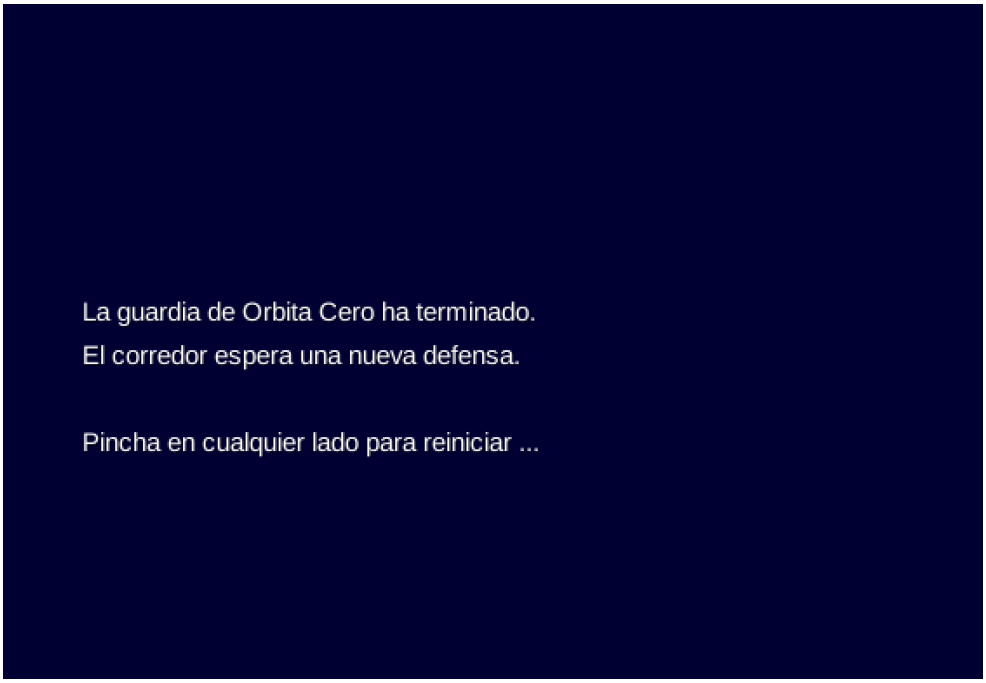


Figura 4. Fin del juego

GM-2 Análisis del Juego y Modificaciones Respecto del Juego Base

Aplicación al Proyecto

- **Juego base**

    - Acoplamiento de responsabilidades: en la versión inicial, la nave leía directamente la entrada, creaba proyectiles y se dibujaba a sí misma. La pantalla coordinaba, además, construcción de entidades, reglas de la ronda, colisiones y transición entre pantallas. Esta concentración dificultaba probar cada regla de forma aislada y sustituir una implementación sin alterar sus clientes.

    - Colisiones y estado de la partida: la pantalla conservaba colecciones redundantes de asteroides. Ante un choque se invertía la velocidad sin separar los objetos superpuestos, por lo que podían permanecer atrapados. La gestión de daño y protección también interfería con la continuidad del movimiento.

    - Combate limitado: los asteroides eran obstáculos pasivos, sin ataques ni variantes de comportamiento. La nave disparaba únicamente hacia arriba. La experiencia no ofrecía decisiones de orientación ni adversarios con resistencia y recompensa diferenciadas.

    - Movimiento y presentación: parte del desplazamiento dependía de cada fotograma y, por tanto, variaba con la frecuencia de actualización. Las pantallas usaban dimensiones inconsistentes, lo que complicaba mantener el área jugable al redimensionar la ventana.

    - Recursos y construcción: se cargaban texturas repetidas para entidades de una misma ronda y no se definía claramente quién debía liberarlas. La preparación de la pantalla acumulaba numerosos parámetros y detalles de creación, con riesgo de duplicación y configuraciones incoherentes.

- **Modificaciones propuestas e implementadas**

    - Responsabilidades y entrada: se separaron el cálculo del movimiento, el estado de salud y la lectura de controles. La nave recibe un contrato de entrada y delega la modalidad de ataque en una estrategia. La pantalla conserva la coordinación de la ronda y recibe los proyectiles mediante un contrato acotado. El audio de la nave se comunica ahora mediante eventos y se conecta con LibGDX al construir la ronda.

    - Espacio y colisiones: se fijó un área lógica común y una vista que adapta su presentación al tamaño de ventana. El desplazamiento utiliza el tiempo transcurrido y normaliza las diagonales. Las colisiones entre enemigos separan los cuerpos superpuestos antes de ajustar sus velocidades.

- Enemigos activos: los asteroides pasivos dieron paso a dos tipos de nave enemiga con comportamiento compartido, ataques dirigidos, resistencia y recompensas propias. La variante artillera modifica su respuesta cuando recibe daño y sobrevive.

- Contrato de colisión: jugador, enemigos y proyectiles comparten una comprobación geométrica. Cada objeto conserva la responsabilidad de su consecuencia: consumir el proyectil, descontar vida, aplicar protección o resolver un rebote. La pantalla coordina la puntuación y el retiro de enemigos destruidos.

- Ataques intercambiables: dos estrategias generan, respectivamente, un proyectil dirigido o una ráfaga abierta. La nave del jugador cambia de modalidad desde la tercera ronda y cada tipo de enemigo recibe la suya al crearse. La elección queda fuera de la lógica común de actualización.

- Recursos y rondas: un gestor comparte y libera los recursos multimedia. Un constructor de rondas recibe una configuración validada y crea la pantalla con sus componentes; un director define los valores de inicio y avance. Así se reduce la carga del constructor de la pantalla.

Cambios observables en la jugabilidad

El juego incorpora dos adversarios: uno se destruye con un impacto y dispara una bala; el otro resiste dos impactos, dispara una ráfaga y acelera su ataque cuando queda con una vida. Sus recompensas son distintas. Los disparos del jugador apuntan al ratón y, desde la tercera ronda, pasan a una ráfaga triple.

El jugador se desplaza con cuatro teclas y puede combinar direcciones. La nave gira hacia el ratón, dispara con clic y dispone de una breve protección tras perder una vida. La protección evita daños sucesivos inmediatos sin detener el control.

El área lógica mantiene sus proporciones al redimensionar. Cada ronda conserva vidas y puntaje, incorpora dos enemigos adicionales y aumenta su velocidad base. Los impactos, choques directos y proyectiles enemigos tienen consecuencias diferenciadas.

La progresión termina cuando se agotan las vidas. La pantalla de juego administra la recompensa al destruir enemigos y decide el paso a la ronda siguiente o a la pantalla de derrota.

Alcance arquitectónico y límite pendiente

Salud, movimiento, entrada, estrategias, creación de rondas y audio de la nave tienen responsabilidades más acotadas. La pantalla sigue concentrando actualización, colisiones, puntaje y flujo. El gestor centraliza la propiedad de recursos, pero la pantalla aún conoce objetos multimedia concretos.

Las entidades del dominio siguen fuertemente acopladas a LibGDX: la nave, los enemigos y los proyectiles almacenan objetos gráficos, exponen operaciones de dibujo y usan tipos geométricos de la biblioteca para sus colisiones. Las estrategias de ataque también reciben una textura para

fabricar proyectiles. La nueva interfaz retira de la nave la dependencia directa de las clases de audio, pero no desacopla el renderizado ni toda la creación de proyectiles. Una separación completa requeriría representar posición, geometría y eventos como datos del dominio y trasladar dibujo, efectos y adaptación de recursos a una capa de presentación. Esa refactorización no se atribuye a la implementación actual.

Evidencia

core/src/main/java/puppy/code/MovimientoNave.java— desplazamiento con delta, diagonales normalizadas y límites (líneas 12–25).

core/src/main/java/puppy/code/PantallaJuego.java— colisiones, puntaje y actualización de enemigos (líneas 67–84).

core/src/main/java/puppy/code/Nave4.java— lectura de controles, movimiento y disparo mediante estrategia (líneas 35–55).

core/src/main/java/puppy/code/PantallaJuegoBuilder.java— construcción de la ronda y selección de estrategia (líneas 38–55).

core/src/main/java/puppy/code/Health.java— actualización de la protección y aplicación del daño (líneas 30–42).

GM-3 Diseño de Clases del Dominio y Correspondencia con Java

Aplicación al Proyecto

Entidades centrales del dominio

El dominio de Órbita Cero se organiza alrededor de PantallaJuego, Nave4, NaveEnemiga y Bullet. PantallaJuego coordina el desarrollo de cada ronda mediante colecciones privadas de enemigos y proyectiles, actualiza las entidades, procesa las colisiones, administra el puntaje y determina el avance entre rondas.
Nave4 representa la nave del jugador y administra su posición, orientación, movimiento y disparos, apoyándose en Health para controlar las vidas y el tiempo de protección después de recibir daño.
NaveEnemiga es una clase abstracta que reúne el comportamiento compartido de los enemigos, incluyendo movimiento, resistencia, colisiones y ataques. NaveCaza y NaveArtillera heredan de ella y presentan diferencias en sus vidas, estrategias de disparo, recompensas y reacciones al daño.
Bullet representa los proyectiles y contiene su velocidad, origen y estado de destrucción. BalaEnemiga especializa su comportamiento para procesar los impactos contra el jugador. Nave4, NaveEnemiga y Bullet implementan la interfaz Colisionable, que proporciona un contrato común para detectar colisiones.

Estado, comportamiento y colaboraciones

EstrategiaDisparo define el comportamiento de disparo mediante las implementaciones DisparoSimple y DisparoTriple. PantallaJuegoBuilder configura la estrategia del jugador según la ronda, mientras cada tipo de enemigo establece la suya.
Las responsabilidades auxiliares se distribuyen entre clases específicas: DireccionDisparo calcula direcciones y ángulos, MovimientoNave determina el desplazamiento y AreaJuego establece los límites del escenario. ColisionAsteroides calcula las separaciones y ajustes de velocidad necesarios para resolver los choques entre enemigos.
La lectura de controles se realiza mediante ControlesNave y su implementación ControlesTecladoNave, mientras EntradaNave representa el estado de los controles. Nave4 utiliza SonidosNave para comunicar eventos de audio, implementados mediante SonidosNaveLibGdx.
GestorRecursos centraliza la obtención y liberación de los recursos multimedia utilizados por el videojuego.

Clases de LibGDX y organización de las pantallas

SpaceNavigation extiende Game y administra el ciclo de vida de la aplicación. PantallaMenu, PantallaJuego y PantallaGameOver implementan Screen y representan los distintos estados visuales del videojuego.
La construcción de rondas se organiza mediante ConstructorPantallaJuego, DirectorRondas, PantallaJuegoBuilder y RondaConstruida. FabricaEnemigosRonda permite crear los distintos tipos de enemigos mediante el contrato FabricaEnemigos.

El diseño también incorpora las clases e interfaces de LibGDX utilizadas en el proyecto, incluyendo componentes de renderizado, cámara, viewport, texturas, sprites, audio, entrada y detección de colisiones. Sus dependencias permiten identificar cómo se relaciona la implementación Java con el motor del videojuego.

Organización y justificación de los diagramas UML

Para representar el diseño del proyecto se decidió elaborar un diagrama UML general que incluye los 34 tipos Java propios de producción, junto con las clases e interfaces externas relevantes y sus relaciones. Esta decisión busca ofrecer una visión completa de la estructura del videojuego y evitar las omisiones de clases y dependencias señaladas en la evaluación anterior.
Sin embargo, debido a la cantidad de elementos y relaciones, el diagrama general resulta extenso y puede dificultar la lectura de algunas conexiones. Por este motivo, se incorporaron seis vistas complementarias, organizadas según las responsabilidades de las clases:

- **Aplicación y pantallas:** ciclo de vida del juego y navegación entre pantallas.
- **Construcción de rondas:** configuración, creación y progresión de las rondas.
- **Entidades y combate:** jugador, enemigos, proyectiles y colisiones.
- **Entrada, movimiento y disparo:** controles, desplazamiento y estrategias de ataque.
- **Recursos y audio:** gestión de recursos multimedia y reproducción de sonidos.
- **LibGDX y lanzador:** clases externas utilizadas y componentes de ejecución del videojuego.

Estas vistas no representan diseños independientes ni reemplazan al UML general. Son representaciones focalizadas de la misma implementación que muestran las clases y relaciones pertinentes a cada área, facilitando su lectura sin eliminar información del modelo completo.
De esta manera, el UML general permite comprobar la cobertura de las clases propias y de LibGDX, mientras que las vistas especializadas facilitan comprender las relaciones de herencia, implementación, asociación y dependencia entre los componentes.

Las capturas presentadas a continuación muestran el UML general y sus seis vistas complementarias. Debido al tamaño del modelo, hay un enlace permite acceder al archivo completo y examinar cada diagrama con mayor detalle.

En la carpeta de Google Drive se encuentra el archivo editable del UML general de GM-3, junto con los diagramas UML correspondientes a los demás requerimientos del proyecto. También se incluye un documento que reúne los enlaces de acceso a todos los diagramas, con el propósito de facilitar su consulta y revisión.

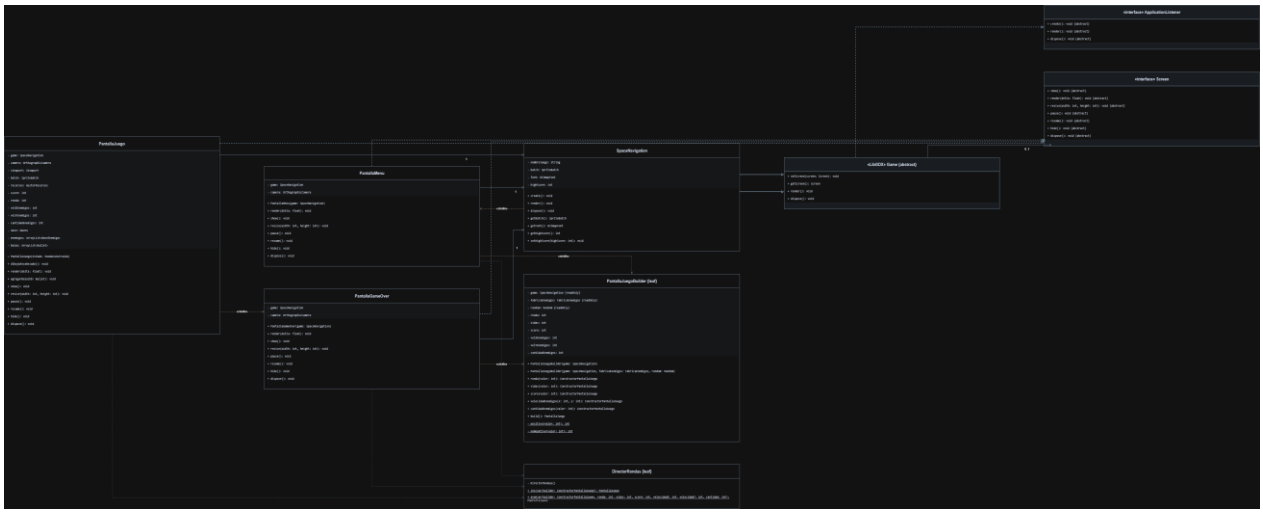
Evidencia

Link a los diagramas UML de GM-3: https://n9.cl/cbta8g
Drive: https://drive.google.com/drive/folders/1zCaEXwwvzaxvqtHMkgRhV43HrZf60moy

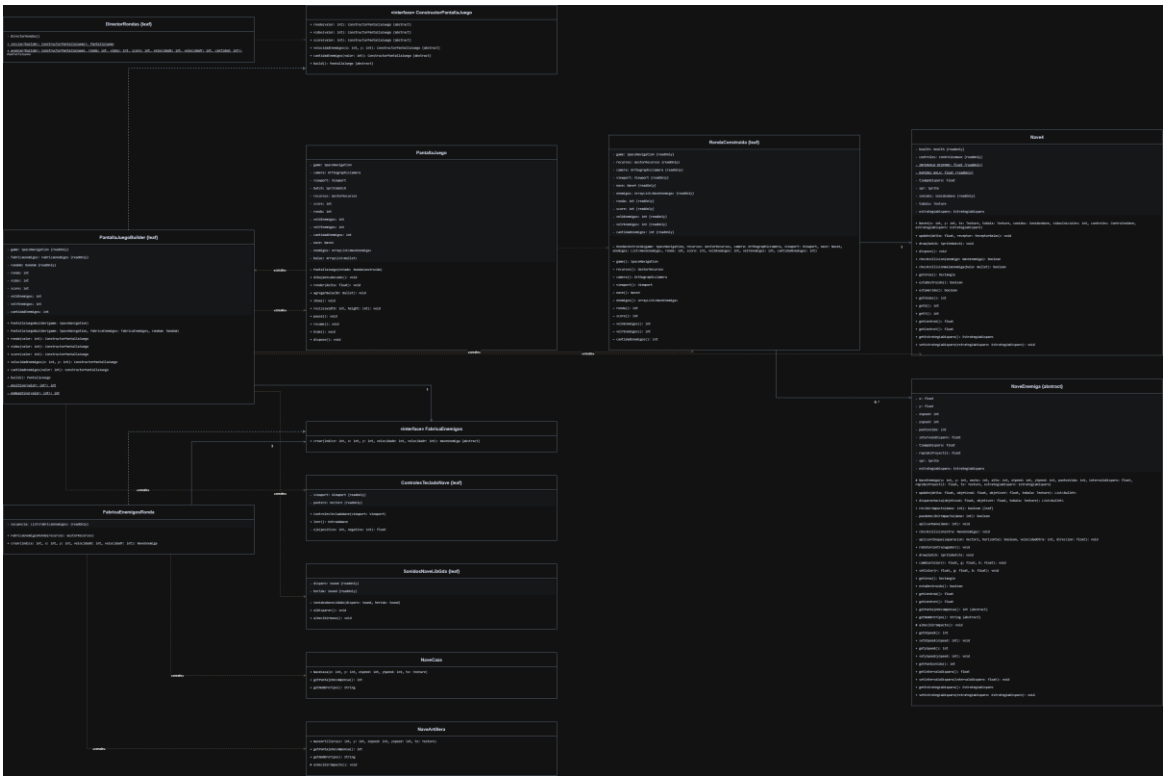
(General)



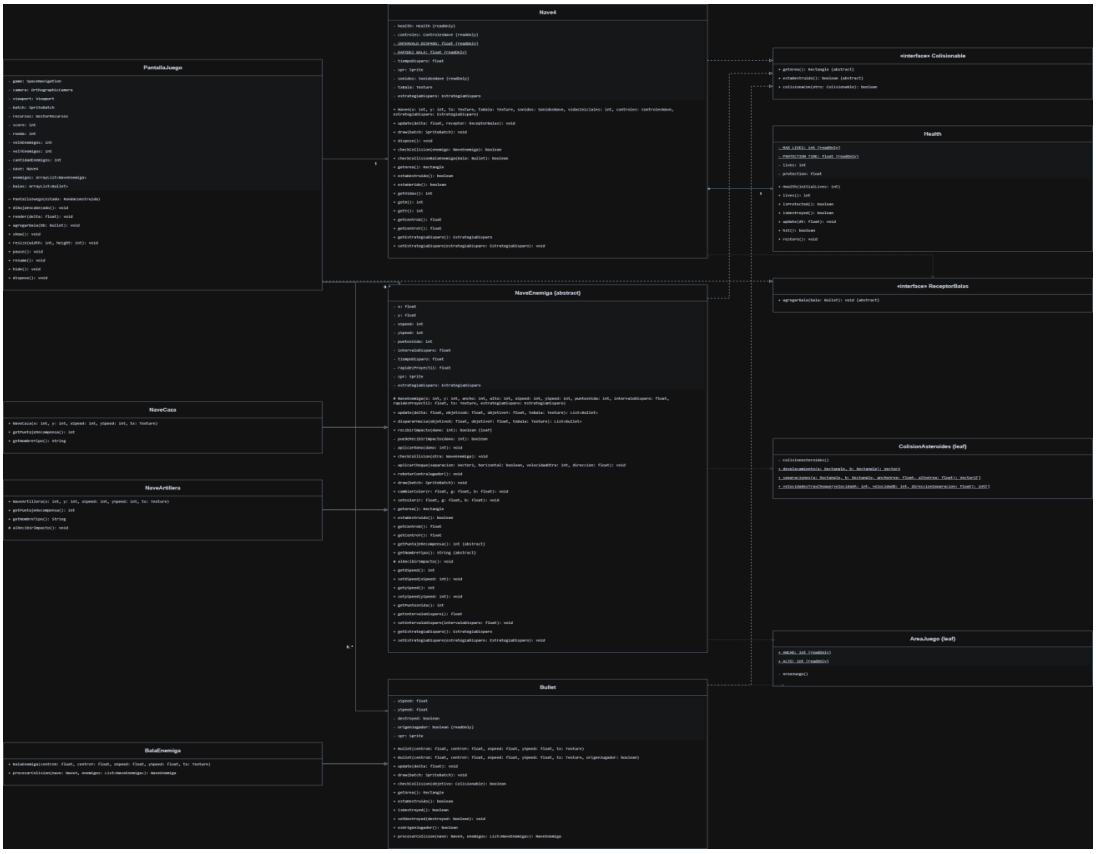
(Aplicación y pantallas)



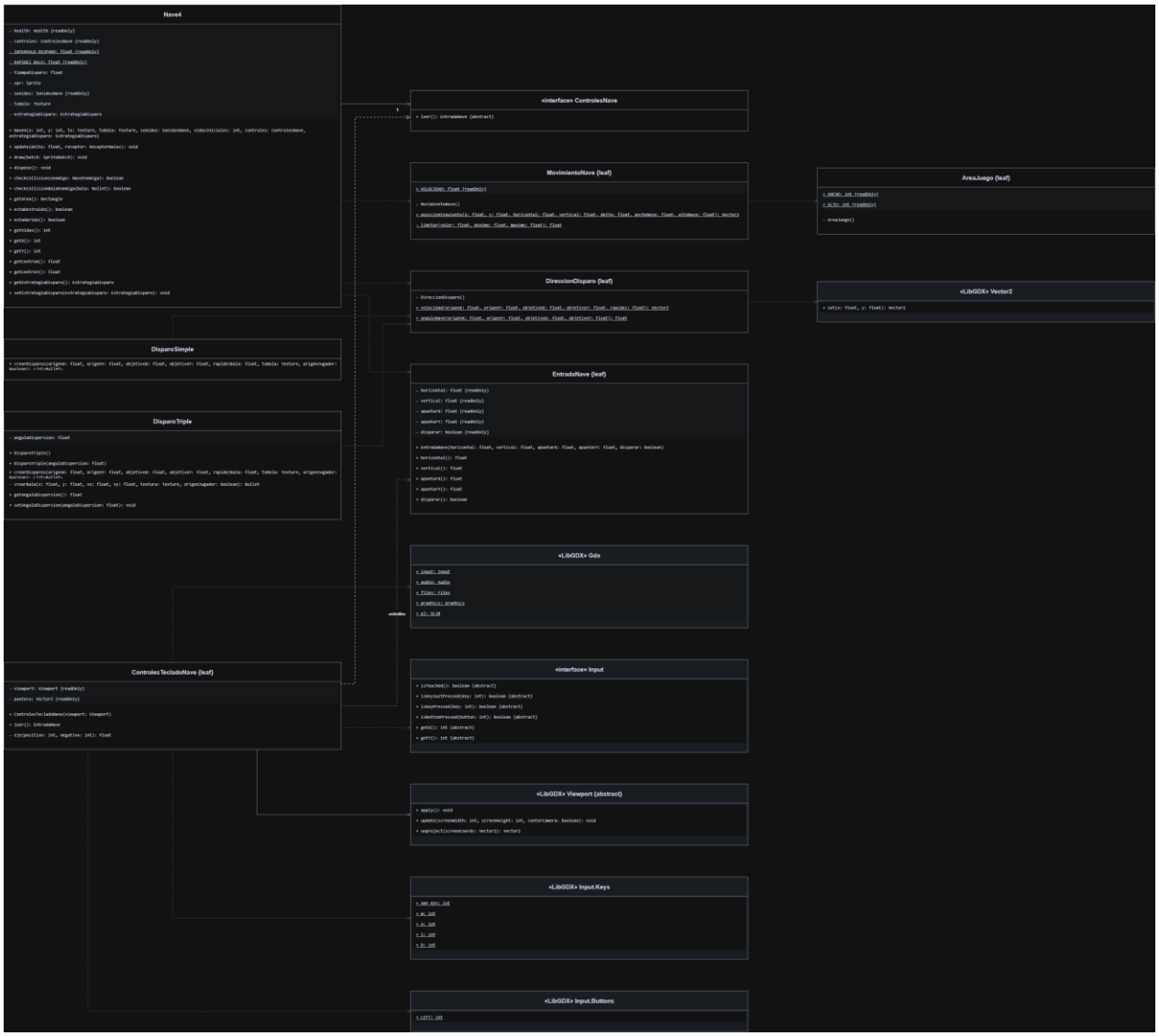
(Construcción de rondas)



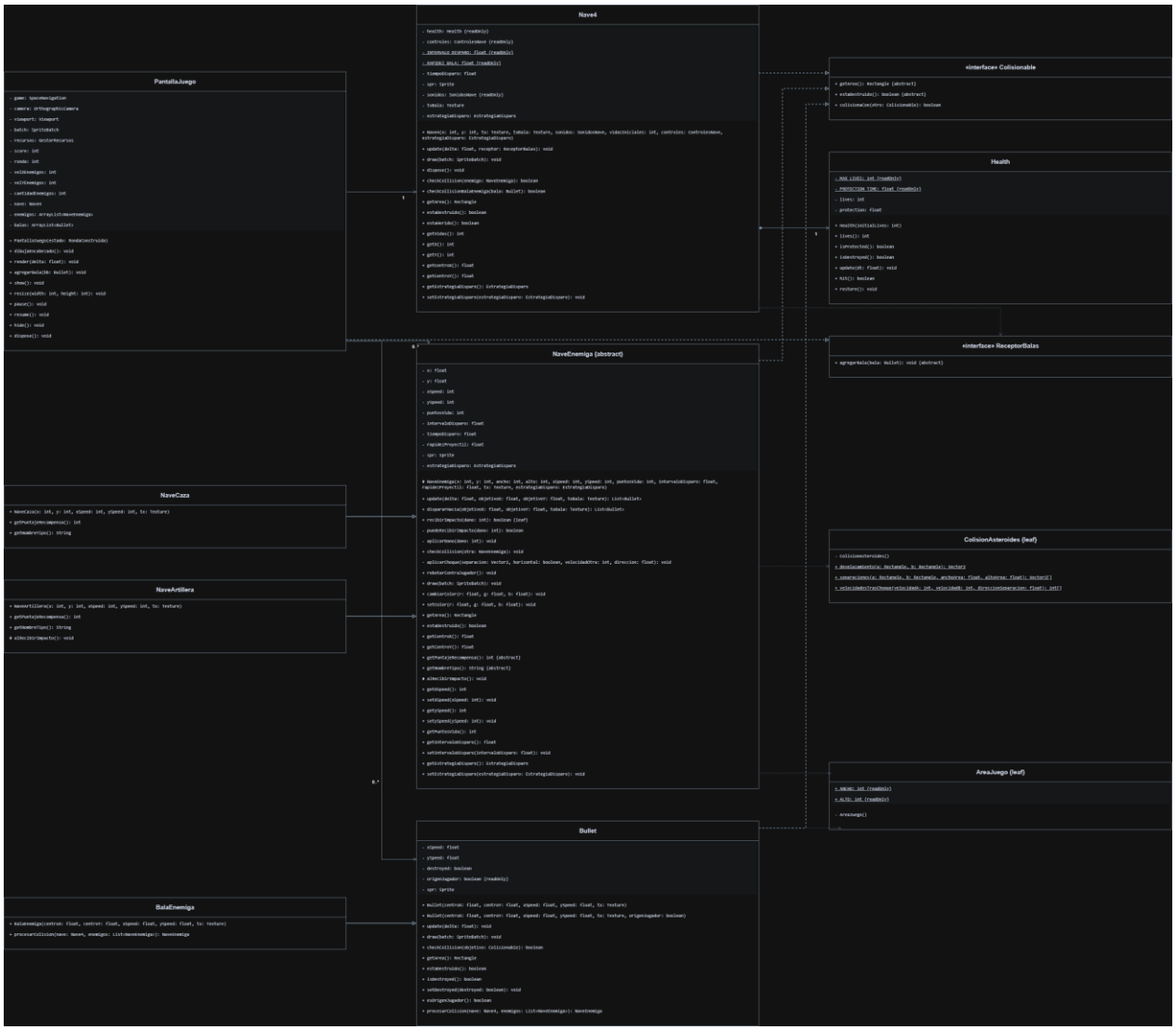
(Entidades y combate)



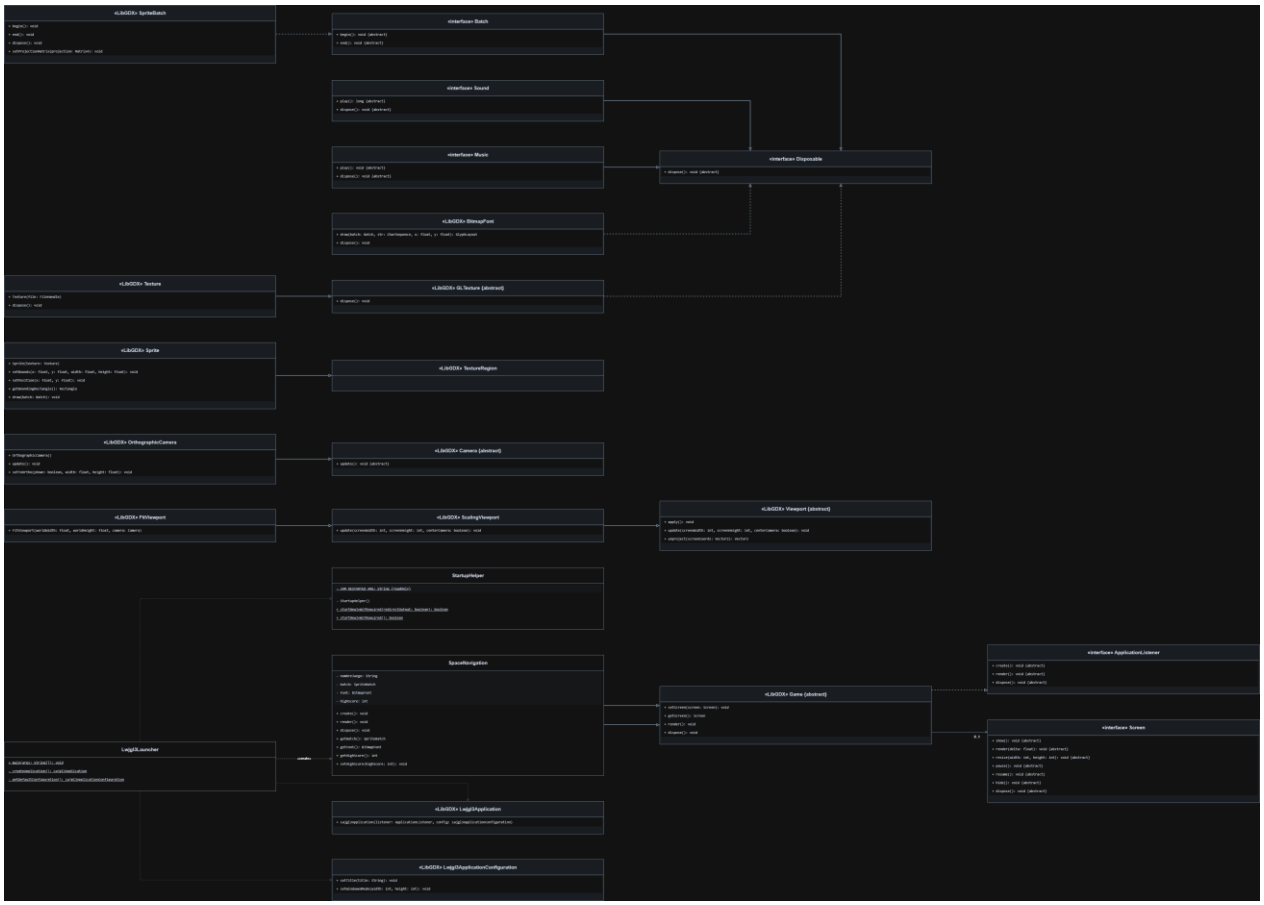
(Entrada movimiento y disparo)



(Recursos y audio)



(LibGDX y lanzador)



GM-4 Clase Abstracta e Interfaz Utilizadas en el Juego

Aplicación al Proyecto

Jerarquía de naves enemigas

NaveEnemiga es una clase abstracta utilizada como tipo común de los enemigos almacenados en PantallaJuego. Su constructor establece posición, dimensiones, velocidad, resistencia, intervalo de ataque, textura y estrategia de disparo. update(...) ejecuta el desplazamiento, la comprobación de límites, la orientación hacia el jugador y la creación periódica de proyectiles. La misma clase también resuelve las colisiones entre enemigos.

NaveCaza y NaveArtillera heredan de NaveEnemiga y son creadas por FabricaEnemigosRonda y almacenadas como NaveEnemiga en PantallaJuego. NaveCaza dispone de una vida, utiliza DisparoSimple y otorga 15 puntos. NaveArtillera tiene dos vidas, utiliza DisparoTriple y entrega 35 puntos; cuando queda con una vida disminuye a la mitad el intervalo entre disparos y cambia de color.

La clase abstracta exige que las subclases implementen getPuntajeRecompensa() y getNombreTipo(). El hook alRecibirImpacto() ofrece una implementación vacía por defecto: NaveArtillera lo especializa y NaveCaza hereda el comportamiento común. Por medio de referencias NaveEnemiga, PantallaJuego puede actualizar, dibujar y procesar impactos de ambos tipos sin realizar comprobaciones de tipo.

Justificación de la clase abstracta: los enemigos comparten estado privado e invariantes de posición, vida, movimiento, disparo y choque, además de una implementación común de update() y recibirImpacto(int). Una interfaz solo declararía un contrato y obligaría a duplicar o delegar ese estado y algoritmo en cada tipo. NaveEnemiga impide crear un enemigo genérico incompleto y exige únicamente recompensa y nombre propios de cada subtipo. El hook opcional alRecibirImpacto() permite que NaveCaza use el comportamiento común sin un método vacío y que NaveArtillera especialice su reacción.

Contrato de colisión implementado por varias clases

Colisionable es la interfaz compartida por Nave4, NaveEnemiga y Bullet. Declara getArea() y estaDestruido() y proporciona el método default colisionaCon(Colisionable), que comprueba existencia, estado activo y superposición de los rectángulos de las entidades.

Justificación del método default: getArea() y estaDestruido() son datos que cada entidad obtiene de su propio estado; colisionaCon(Colisionable) combina esos datos con una regla geométrica idéntica para Nave4, NaveEnemiga y Bullet. Su implementación default evita repetir la comprobación de null, destrucción y solapamiento en tres clases sin imponer herencia de una segunda clase base. El contrato solo detecta contacto; cada objeto conserva la responsabilidad de decidir el efecto. NaveEnemiga.checkCollision(NaveEnemiga) ahora consulta colisionaCon() antes de calcular separación y rebote.

Las consecuencias se resuelven en cada objeto: Bullet se destruye al impactar; Nave4 administra pérdida de vidas y protección; NaveEnemiga recibe daño y resuelve rebotes con otros

adversarios. PantallaJuego utiliza estas operaciones en el procesamiento efectivo de la ronda, por lo que la clase abstracta y la interfaz poseen un uso concreto en el sistema.

Alcance del diagrama UML

El diagrama UML de GM-4 se centra en las relaciones de herencia de NaveEnemiga y en las clases que implementan Colisionable. Los métodos abstractos se representan en cursiva, mientras que las características final y default se explican en el reporte sin añadir anotaciones al diagrama. Se omite PantallaJuego, aunque utiliza estas clases, para mantener el enfoque en la estructura y el comportamiento de la clase abstracta y la interfaz.

Evidencia

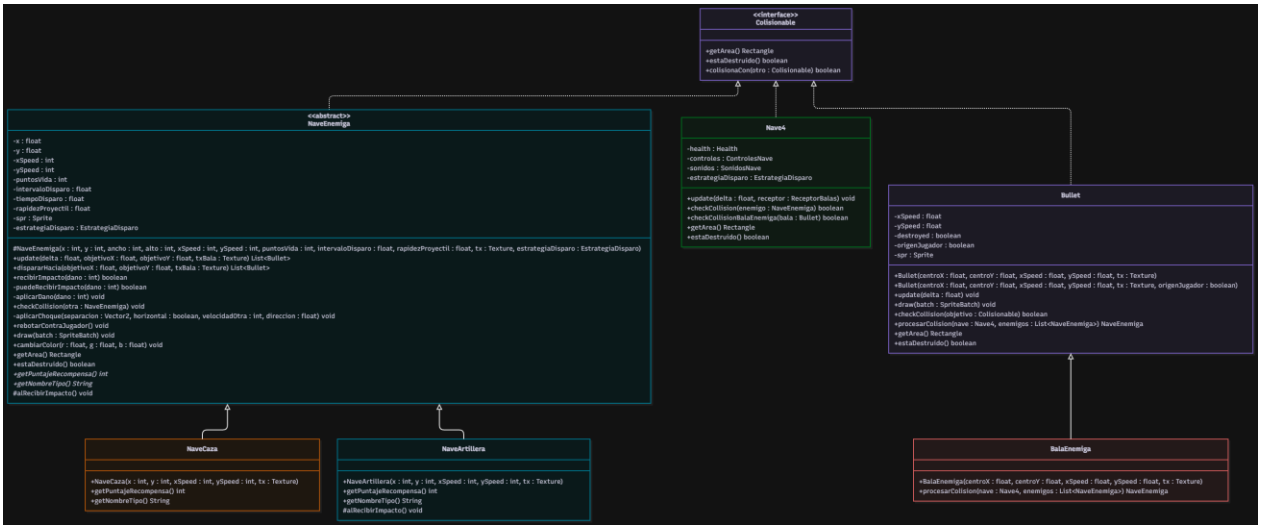
core/src/main/java/puppy/code/NaveEnemiga.java— métodos abstractos y hook de impacto (líneas 163–167).

core/src/main/java/puppy/code/Colisionable.java— interfaz y método default de colisión (líneas 6–18).

core/src/main/java/puppy/code/NaveEnemiga.java— clase abstracta con estado privado y constructor compartido (líneas 12–38).

core/src/main/java/puppy/code/NaveCaza.java— especialización con disparo simple y recompensa (líneas 4–19).

core/src/main/java/puppy/code/NaveArtillera.java— especialización con disparo triple y reacción al impacto (líneas 6–28).



Link: https://n9.cl/5e2tx
Drive: https://drive.google.com/drive/folders/1zCaEXwwvzaxvqtHMkgRhV43HrZf60moy

GM-5 Encapsulamiento y Principios de Orientación a Objetos

Aplicación al Proyecto

Protección del estado y operaciones de dominio

PantallaJuego conserva privadas sus colecciones de enemigos y balas. NaveEnemiga.update(...) devuelve los disparos producidos y PantallaJuego los incorpora a su lista; el enemigo no recibe ni modifica la colección interna de la pantalla. Nave4 publica disparos a través del contrato ReceptorBalas.

Nave4 mantiene privados Health, los controles, Sprite y la estrategia de disparo, y recibe EstrategiaDisparo por constructor. Health.hit() valida los impactos y aplica un segundo de protección. NaveEnemiga conserva privados sus datos de combate: cada instancia aplica a su propio estado la separación y velocidad del choque mediante aplicarChoque(...), con límites y posición del Sprite sincronizados. DisparoTriple valida el ángulo de dispersión.

Responsabilidades y cohesión: GRASP

Health administra las vidas y su protección; MovimientoNave calcula el desplazamiento; NaveEnemiga administra resistencia, ataque y su respuesta de choque. PantallaJuego coordina la ronda y conserva las colecciones. FabricaEnemigosRonda configura la secuencia de tipos; PantallaJuegoBuilder usa esa fábrica al generar los enemigos.

GestorRecursos reúne la carga, entrega y liberación de texturas, sonidos y música, evitando repetir esa lógica en cada pantalla. El acoplamiento de la entrada disminuye porque Nave4 utiliza la interfaz ControlesNave y el objeto EntradaNave en lugar de consultar directamente el teclado.

Aplicación concreta de SOLID

La responsabilidad única (SRP) se favorece al separar Health, MovimientoNave, GestorRecursos, las estrategias de disparo y FabricaEnemigosRonda. La apertura a extensión (OCP) permite suministrar otra FabricaEnemigos sin modificar la selección de enemigos en PantallaJuego. La pantalla delega la colisión de cada proyectil en Bullet.procesarColision(...); BalaEnemiga especializa ese comportamiento.

La sustitución de Liskov (LSP) se refleja en el tratamiento de NaveCaza y NaveArtillera mediante referencias NaveEnemiga, y de BalaEnemiga mediante referencias Bullet. La segregación de interfaces (ISP) se favorece con contratos acotados como Colisionable, ControlesNave, ReceptorBalas y FabricaEnemigos. La inversión de dependencias (DIP) se aplica cuando Nave4 recibe EstrategiaDisparo y usa ReceptorBalas en update(...), y PantallaJuego recibe FabricaEnemigos. Se mantienen dependencias concretas de LibGDX para presentación y recursos.

Evidencia

core/src/main/java/puppy/code/Nave4.java— campos privados y recepción de ControlesNave y EstrategiaDisparo (líneas 12–27).

core/src/main/java/puppy/code/PantallaJuego.java— colecciones privadas de enemigos y proyectiles (líneas 30–32).

core/src/main/java/puppy/code/Health.java— validación de vidas y protección del estado (líneas 10–42).

core/src/main/java/puppy/code/NaveEnemiga.java— recepción de impactos con validación y cambio interno de vida (líneas 70–84).

core/src/main/java/puppy/code/DisparoTriple.java— validación del ángulo de dispersión en el setter (líneas 54–59).

GM-6 Aplicación de diseño Singleton

Aplicación al Proyecto

Problema

En el juego base, cada pantalla y entidad podía cargar por separado archivos multimedia que también utilizaban otros objetos. Al crear una ronda nueva, volver a abrir las mismas texturas, sonidos y música aumentaba el uso de memoria y dispersaba la responsabilidad de liberarlos. Si una pantalla se cerraba mientras otra seguía utilizando un recurso compartido, liberar ese recurso desde la pantalla podía dejar referencias inválidas; si nadie lo liberaba, permanecía ocupado hasta terminar el juego.

Contexto

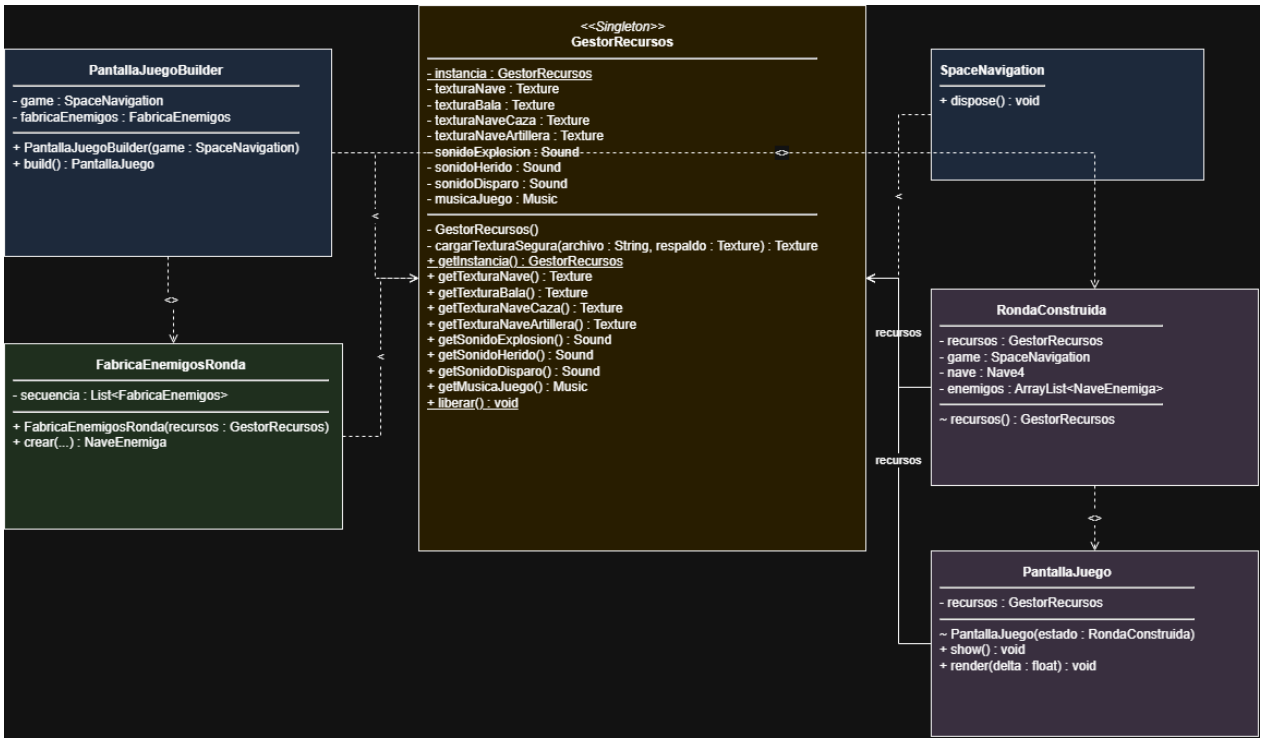
Órbita Cero crea una PantallaJuego por ronda y comparte recursos entre la nave del jugador, las naves enemigas y las pantallas. GestorRecursos se inicializa cuando un consumidor solicita por primera vez getInstancia(); su constructor privado carga texturas, sonidos y música. PantallaJuegoBuilder obtiene la misma instancia para construir la ronda y la pasa a los objetos que necesitan recursos. Las texturas alternativas de enemigos usan la textura principal como respaldo cuando falta el archivo. El gestor conserva la propiedad de los recursos cargados; los consumidores los utilizan sin asumir su liberación.

Solución

GestorRecursos aplica Singleton mediante una instancia estática privada, constructor privado y getInstancia() público. Así, las rondas reciben el mismo gestor y no repiten la carga. Los getters entregan referencias a los recursos necesarios y mantienen la creación dentro del gestor. SpaceNavigation llama a GestorRecursos.liberar() al cerrar el juego: el método libera cada textura propia, sonido y música, evita liberar dos veces la textura principal cuando se usó como respaldo y restablece la instancia a null. Una solicitud posterior puede crear una instancia nueva. Esta decisión concentra la propiedad y el ciclo de vida, aunque exige que ningún consumidor use referencias anteriores después del cierre.

- Instancia única durante el ciclo activo: getInstancia() devuelve la misma referencia hasta llamar a liberar().
- Acceso controlado: el constructor no está disponible para las pantallas ni las entidades; los recursos se obtienen mediante getters.
- Liberación centralizada: SpaceNavigation solicita el cierre y GestorRecursos dispone únicamente los recursos que posee.

Evidencia



Link:
Drive:

GM-7 Patrón de Diseño Strategy

Aplicación al Proyecto

Problema y contexto

El combate requiere varias modalidades de ataque. NaveCaza y el jugador durante las dos primeras rondas disparan un proyectil por activación, mientras NaveArtillera dispara tres. A partir de la tercera ronda, la nave del jugador también utiliza una ráfaga triple. Concentrar el cálculo de cada trayectoria en las naves produciría condicionales y duplicación del comportamiento de ataque.

Solución y estrategias concretas

La interfaz EstrategiaDisparo define la creación de proyectiles a partir del origen, el objetivo y la rapidez. DisparoSimple produce un proyectil dirigido al objetivo. DisparoTriple produce uno central y dos con trayectorias desviadas en sentidos opuestos; su dispersión predeterminada es de 16 grados. Ambas implementaciones devuelven los proyectiles al objeto que solicitó el disparo.

Nave4 conserva una referencia privada al contrato EstrategiaDisparo y delega en ella la creación de sus proyectiles. PantallaJuegoBuilder inyecta DisparoSimple en las rondas 1 y 2 y DisparoTriple desde la ronda 3, al construir cada nueva pantalla. Así cambia el comportamiento entre rondas sin modificar la lógica de actualización de Nave4.

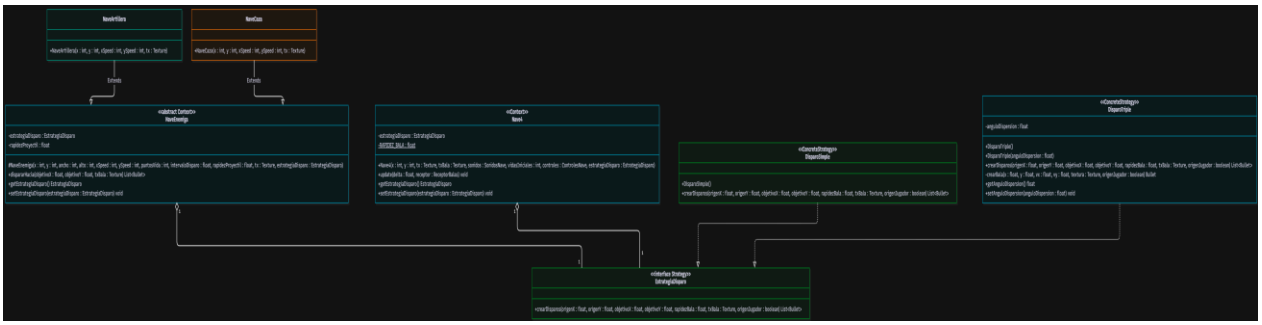
Relación con las clases Java

NaveEnemiga también recibe una EstrategiaDisparo por constructor. NaveCaza usa DisparoSimple y NaveArtillera usa DisparoTriple. Al disparar, la clase base delega la creación de los proyectiles en esa referencia. El contrato compartido permite incorporar nuevas modalidades sin modificar la lógica de disparo de las naves.

Alcance del diagrama UML

El diagrama se centra en las clases y relaciones que permiten comprender la aplicación del patrón Strategy. Aunque PantallaJuegoBuilder selecciona e inyecta la estrategia de disparo del jugador según la ronda, se omite para mantener el enfoque en los contextos y las estrategias concretas. Su funcionamiento se describe en este apartado y su responsabilidad de construcción se aborda en GM-9.

Evidencia



Link: https://n9.cl/bq7z19
Drive: https://drive.google.com/drive/folders/1zCaEXwwvzaxvqtHMkgRhV43HrZf60moy

GM-8 Patrón de Diseño Template Method

Aplicación al Proyecto

Problema y contexto

NaveCaza y NaveArtillera comparten la recepción de daño, pero solo la artillera requiere una reacción adicional cuando sobrevive con una vida. El problema era mantener la validación, el descuento de vida y la determinación de destrucción en un orden estable, sin duplicarlos ni obligar a cada subtipo a implementar una reacción vacía.

Estructura del método plantilla

La solución es recibirImpacto(int dano), método plantilla public final de NaveEnemiga. Ejecuta en orden: puedeRecibirImpacto(dano), que rechaza daño no positivo o una nave destruida; aplicarDano(dano), que reduce puntosVida sin bajar de cero; alRecibirImpacto(), hook protegido opcional; y estaDestruido(), que determina el resultado devuelto. Los pasos de validación y aplicación son privados para proteger la invariante de vida y el orden del algoritmo. Solo el hook permite especializar la reacción.

NaveCaza hereda la implementación vacía de alRecibirImpacto() y se destruye al primer daño válido. NaveArtillera sobrescribe el hook: al quedar con una vida reduce a la mitad su intervalo de disparo y cambia de color. Un impacto inválido no ejecuta el hook; después de la destrucción, los impactos posteriores tampoco alteran el estado.

El método plantilla es final para impedir que una subclase cambie la secuencia o el significado del valor devuelto. La especialización se limita a la reacción, mientras el estado puntosVida permanece privado en NaveEnemiga. El algoritmo permite agregar nuevos tipos de enemigo con otra reacción sin modificar la recepción común, y ambos subtipos conservan el mismo contrato de impacto y destrucción.

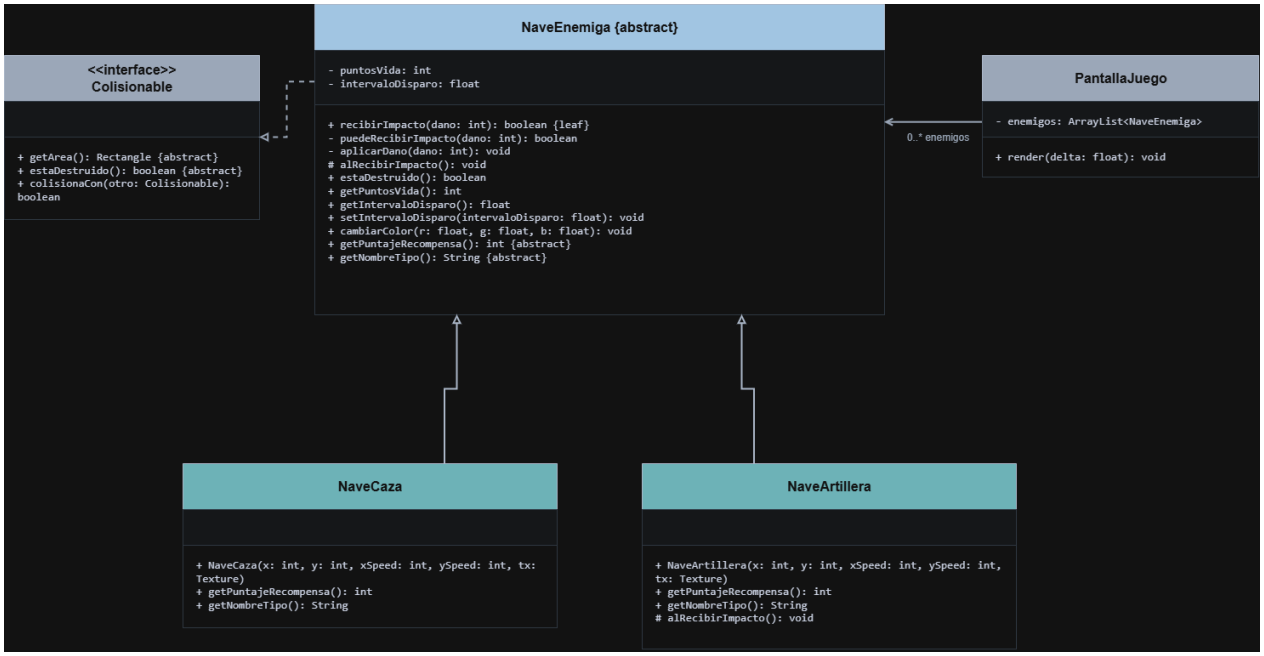
Relación con las clases Java

PantallaJuego invoca recibirImpacto(1) cuando una bala del jugador alcanza una NaveEnemiga y usa el resultado booleano para sumar la recompensa y retirarla si fue destruida. En el código, NaveCaza y NaveArtillera se crean mediante FabricaEnemigosRonda y se procesan como NaveEnemiga. La evidencia del patrón está en NaveEnemiga.recibirImpacto(int), sus pasos privados y el hook alRecibirImpacto() especializado solo por NaveArtillera,

Alcance del diagrama UML

El diagrama UML se focaliza en la implementación del patrón Template Method, por lo que se representan principalmente los atributos, métodos y relaciones necesarios para comprender su funcionamiento. Se prioriza el método plantilla recibirImpacto(int dano), sus pasos internos y el método de extensión alRecibirImpacto(), junto con las clases que participan en este comportamiento. Se omiten otras operaciones de las clases que no aportan directamente a la explicación del patrón, manteniendo el diagrama centrado en el diseño implementado.

Evidencia



Link: https://n9.cl/4s3ssz
Drive: https://drive.google.com/drive/folders/1zCaEXwwvzaxvqtHMkgRhV43HrZf60moy

GM-9 Aplicación del patrón de diseño Builder

Aplicación al Proyecto

Problema

La creación de PantallaJuego requiere configurar la ronda, las vidas, el puntaje y la dificultad, además de preparar cámara, viewport, nave y enemigos.

Antes, varios parámetros enteros hacían difícil distinguir sus significados y el constructor de la pantalla acumulaba la construcción de todos los componentes. Este acoplamiento dificultaba revisar y extender la preparación de una ronda.

Una variación en la generación de enemigos o en la configuración visual obligaba a modificar la pantalla, aunque su responsabilidad principal es ejecutar la ronda.

Contexto

En Órbita Cero se crea una nueva PantallaJuego al iniciar una partida, reiniciar después de un Game Over y avanzar a una nueva ronda.

Además, al avanzar de ronda se deben conservar algunos valores, como las vidas y el puntaje, mientras que otros cambian, como la cantidad y velocidad de los enemigos.

Debido a esto, la creación de PantallaJuego necesita distintas configuraciones dependiendo del momento del juego.

Solución

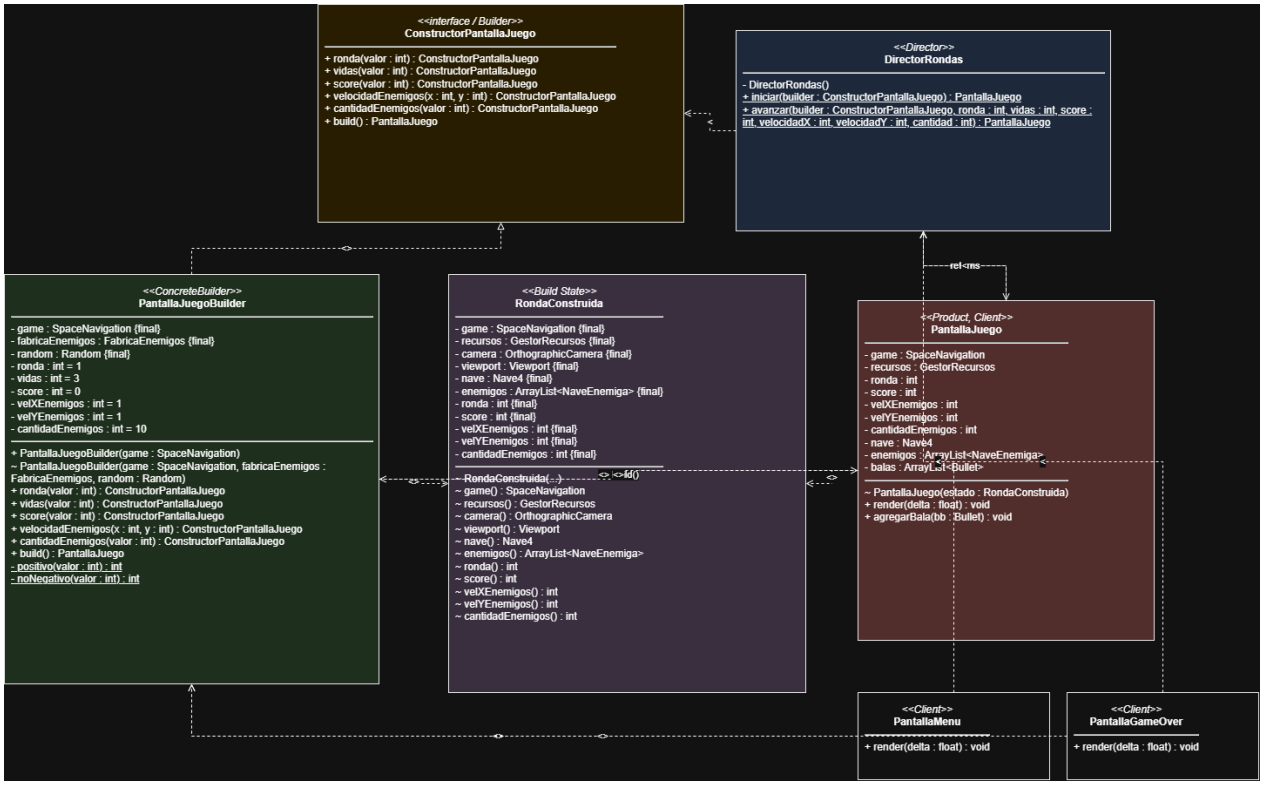
Se separaron las responsabilidades: ConstructorPantallaJuego declara las operaciones de configuración y la entrega del producto; DirectorRondas define las recetas de partida nueva y avance; PantallaJuegoBuilder implementa el contrato y construye los componentes concretos.

La receta de inicio establece la primera ronda con tres vidas, puntaje cero, velocidad inicial de uno en cada eje y diez enemigos. La receta de avance conserva vidas y puntaje, incrementa la ronda y ambas velocidades en uno, y agrega dos enemigos.

PantallaMenu y PantallaGameOver solicitan al Director la receta de inicio. PantallaJuego actúa como cliente al avanzar de ronda y entrega las vidas, el puntaje y la dificultad actuales. El Director depende del contrato ConstructorPantallaJuego, mientras el Builder concreto crea el producto.

El Builder prepara la cámara ortográfica, ajusta el viewport al tamaño de la ventana, crea Nave4 con los controles y el tipo de disparo correspondiente, y genera enemigos mediante FabricaEnemigos. Solo después entrega a PantallaJuego un estado de ronda completo.

Evidencia



Link:
Drive:

GM-10 Utilización de Github

Aplicación al Proyecto

El proyecto utiliza el repositorio giumartinez/orbita-cero para registrar su evolución. La rama main integra aportes realizados en ramas de trabajo y supera el mínimo de seis commits indicado para GM-10. El historial permite reconocer cambios en movimiento, encapsulamiento, recursos, enemigos y patrones de diseño.

Evidencia

Repositorio: https://github.com/giumartinez/orbita-cero
Historial main: https://github.com/giumartinez/orbita-cero/commits/main